Lecture 13 : Designing YouTube | video Streaming platform

Functional Requirements :

1. User can upload a video (Creator)
2. User can view / stream a video

Not Covered :

1. Likes / disliked
2. Channel management
3. Comments

Non functional Req :

1. prefer Availability over consistency

Q. Are there compression internally ?

yes.

Q. CAP? Consistency Vs Availibility
↳ Availibility

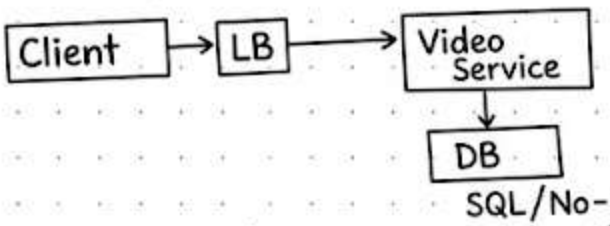
Q. Any restriction on video size?
↳ No restriction. File can be in GBs

Q. DAU
↳ 100 million DAU

→ 1 million create a videos uploded /day
→ 100 million videos are watched /day
→ on avg: 1 user → 1 video watch/day

↳ Back of the envelope calculation:
QPS:
DAU: 100 million
Read QPS: 10 mill videos watch/day
Write QPS: 1 mill videos uploaded /day
Storage Unit:
Avg. video size (After compression): 512 MB

↳ Basic Flow:
1. Posting a video

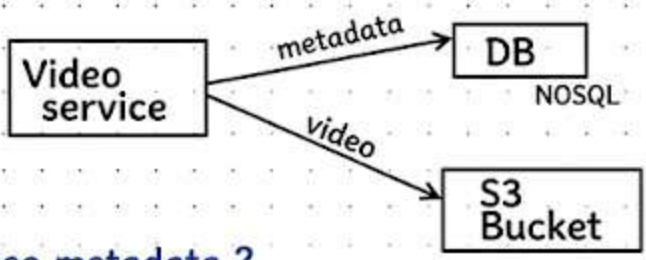


SQL → BLOB (Binary large object)
videos, photos
→ Theoretical possible but we should not

Q. Should we upload in DB?
↳ No

Reasons :

↳ DBs are expensive and its memory become full
↳ DB will become slow
↳ Query not possible on video
↳ Migration (SQL → NOSQL)



Video metadata ?
Video Id,
VideoName,
Desc,

Sample Metadata ?

```
{
    VideoId : 123
    VideoName : "something",
    Video Desc : "Anything",
    timestamps : []

}
```

NoSQL → MongoDB, Cassandra

↳ API End points:
user can upload a video
user can watch a video

1. Uploading a video:
Post: /upload

```
{ video,
  VideoMetaData,
}
```

Response: 200 ok

2. watch / stream a video :

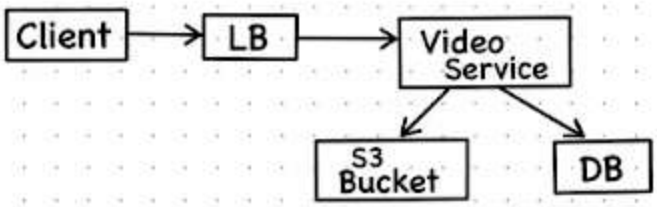
Get : /video/{VideoId}

Response :

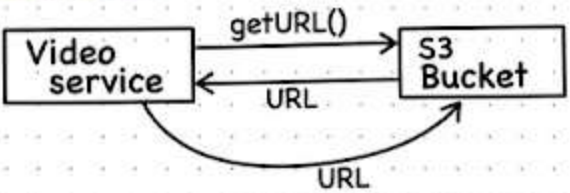
{ video,
VideoMetaData
}

our video

1. Compression on our video
2. 4K → 240p, 480p, 720p, 1080p



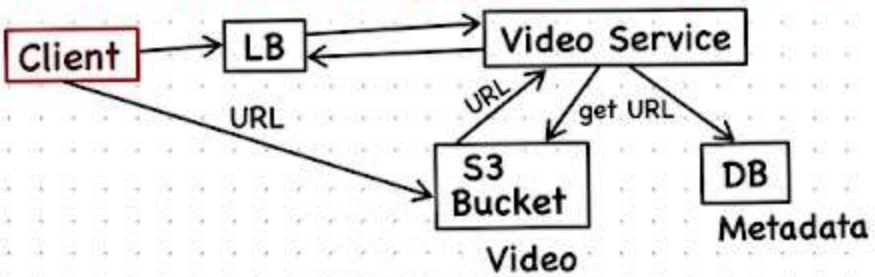
Amazon S3:



Problems:

1. Server will become extremely busy
2. Even if you introduce MQ & workers,
   MQ → limited storage
   This idea will not work

Client Should be responsible for uploading video



Response:
200 ok
URL (to upload the video)

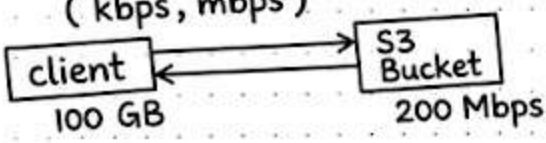
problems:
1. No compression is used
2. [✗ No video chunks]

Concepts:

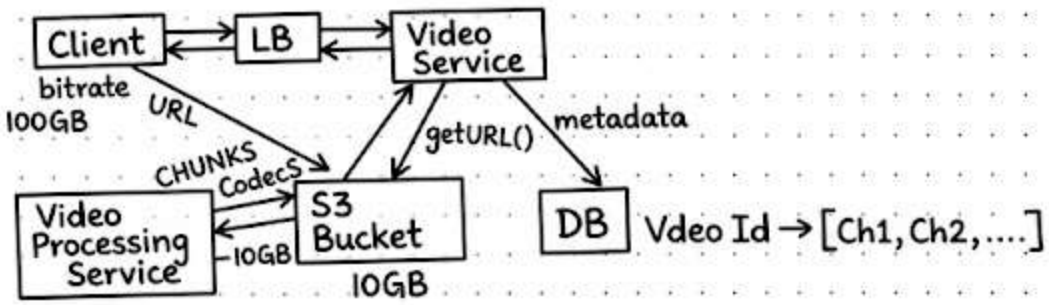
1. Video Codec: e.g.: MPEG-2, MPEG-4, h.265
   Codec : Encoder - Decoder
   - Responsible to compress and decompress a video.
   - Reduce size
   - Try that Quality will remain same.
   - Lossless compression
   - can also do Lossy compression

Trade off
High compression → High chance for lossy compression.

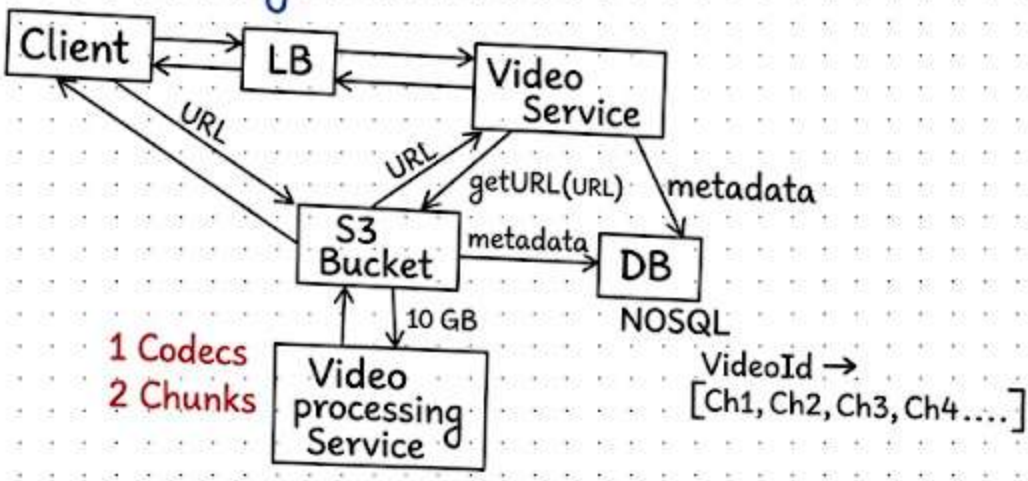
2. Bitrate
   No. of bits transmitted at a given time
   (kbps, mbps)

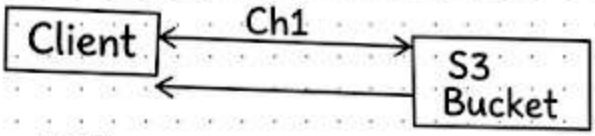


Better Approach: (POST)

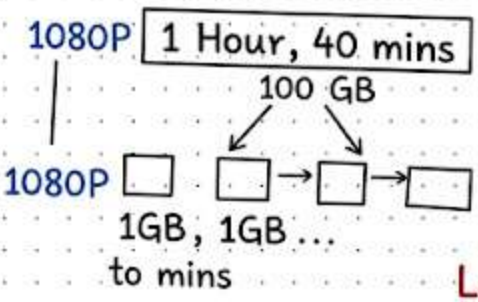


For streaming





FTP ⟶ SFTP
secured file Transferred protocol



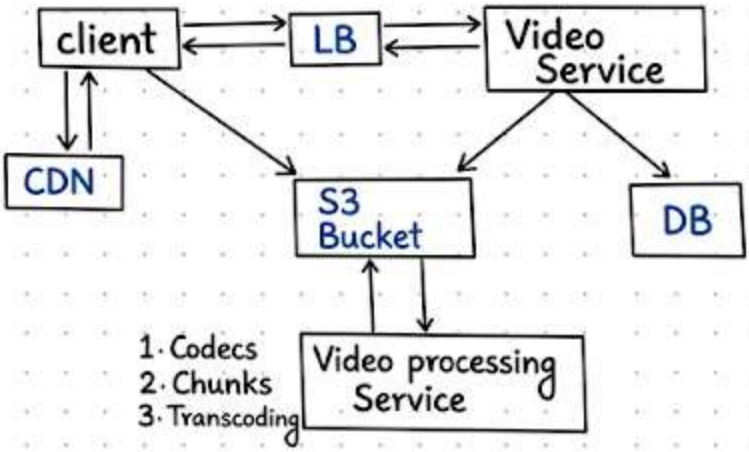
Lossless compression

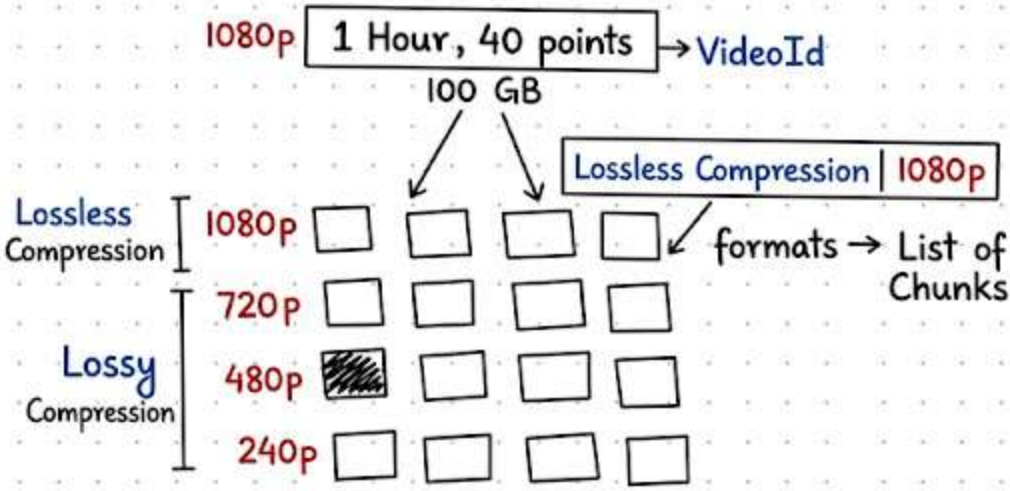
Problems:

Slow Internent connection can lead to too much buffering.

Flexible system

Client → 240p, 480p, 720p, 1080p, ...





Automatic Format Selection

↓

Adaptive Bitrate Streaming

Select (1080p , 480p ....) based on user network

What is Client's downloading speed?

100 Mbps + → 1080p

50 Mbps → 720p